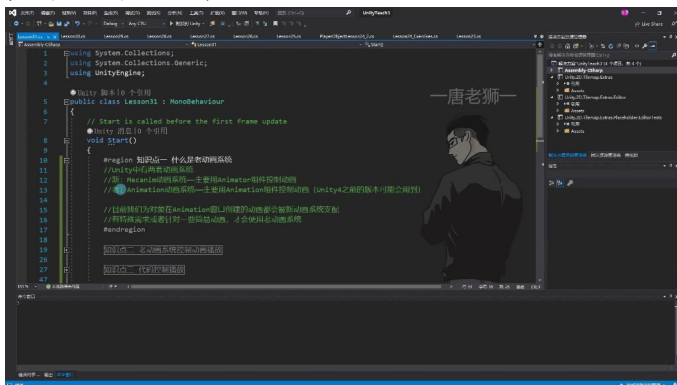


一、代码控制动画老动画系统 00:04

1. 老动画系统 00:19

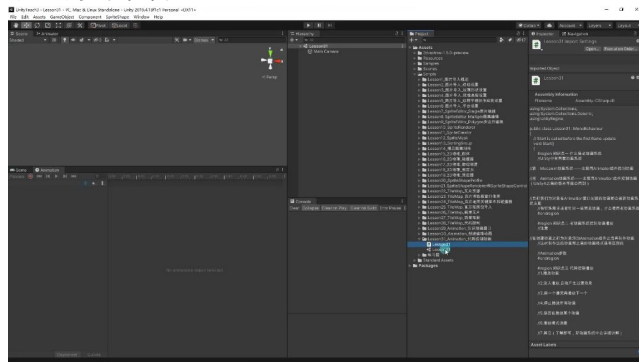
1) 什么是老动画系统 00:33



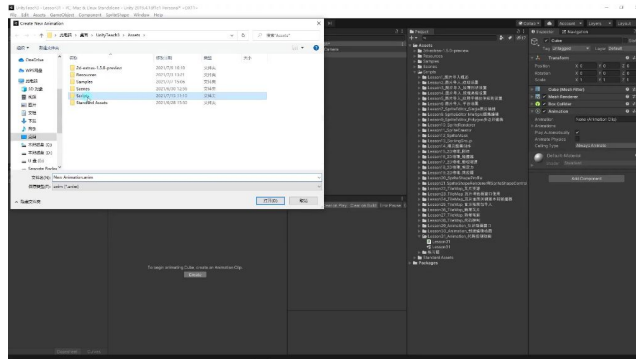
- 两套系统：Unity中存在新旧两套动画系统
 - 新系统：Mecanim动画系统，使用Animator组件控制动画
 - 老系统：Animation动画系统，使用Animation组件控制动画（Unity4之前版本常用）
- 应用场景：
 - 当前Animation窗口创建的动画默认由新系统支配
 - 特殊需求或简易动画才会使用老动画系统

2) 老动画系统控制动画播放 01:22

● 创建动画前的准备 02:24

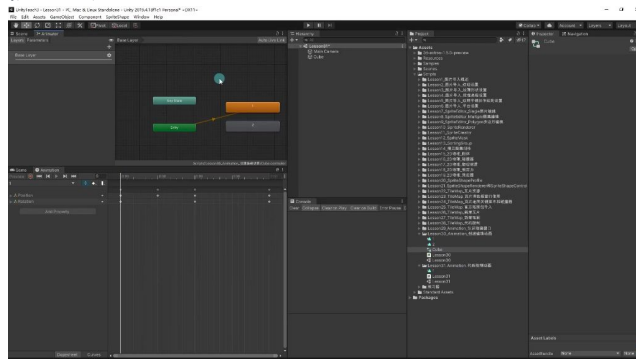


- 关键步骤：
 - 必须为对象先添加Animation组件
 - 之后制作的动画格式将与新系统有本质区别
- 操作演示：
 - 创建Cube对象并重置位置
 - 将摄像机改为透视模式（3D项目标准配置）
- 动画文件的保存 02:47



-
- 保存规范：
 - 动画文件需单独保存（示例中保存为"1.anim"）
 - 建议按课程分类存放（如Lesson31文件夹）

● 系统核心区别 03:03



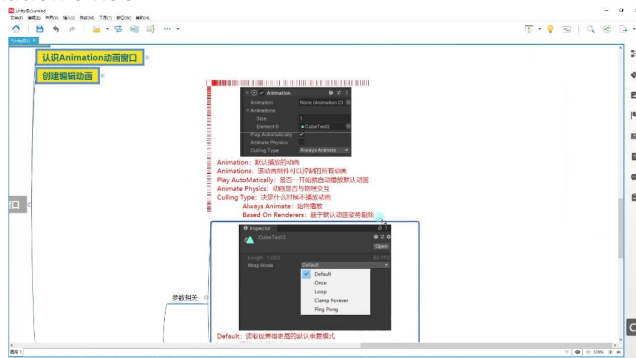
-
- 文件结构差异：
 - 老系统：仅生成.anim动画文件，不产生状态机文件
 - 新系统：同时生成动画文件和Animator Controller文件
- 组件关联：
 - 老系统：通过Animation组件直接关联动画剪辑
 - 新系统：通过Animator组件间接控制动画状态机

● 参数特性说明 04:13

- 编辑共性：
 - 动画曲线编辑方式与新系统完全一致
 - 支持相同的关键帧录制功能（示例演示了Y轴位移动画）
- 核心差异点：
 - Inspector窗口显示的动画参数格式不同
 - Animation组件提供独特的播放控制参数
 - 老系统动画剪辑的元数据配置项更简单直接

3) Animation动画参数 04:31

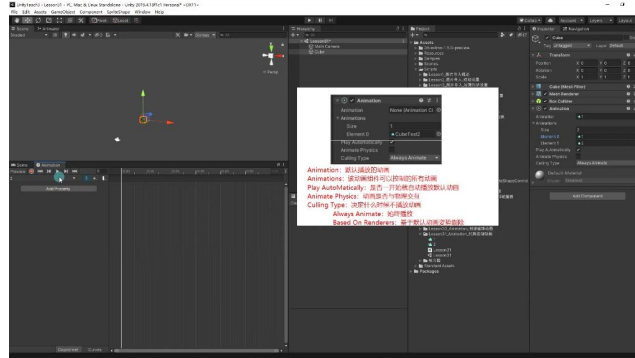
● animation 04:56



-

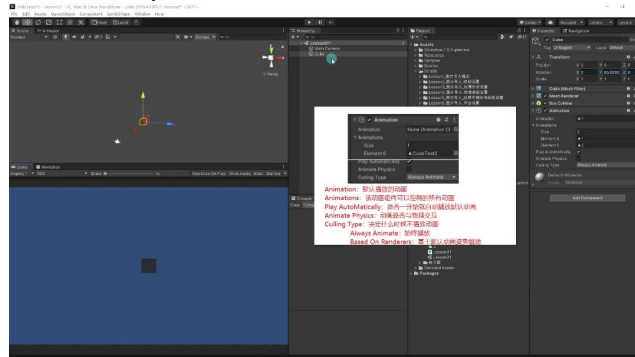
- 定义：Unity中的老动画系统组件
- 新旧系统区别：
 - 新系统：Mecanim动画系统，使用Animator组件控制
 - 老系统：Animation组件控制（Unity4之前版本常用）
- 使用注意：
 - 创建动画前必须先添加Animation组件
 - 默认播放动画需要拖拽老动画系统文件到该参数栏
 - 不设置时运行不会播放任何动画

● animations 05:44



- 功能：记录该对象可控制的所有动画
- 特点：
 - 每创建一个新动画会自动添加到此列表
 - 必须在此关联后才能通过代码控制动画切换
- 示例：
 - 创建旋转90度的动画"二"后，列表显示两个动画
 - 可通过代码在动画"一"和"二"之间切换

● play Automatically 06:47

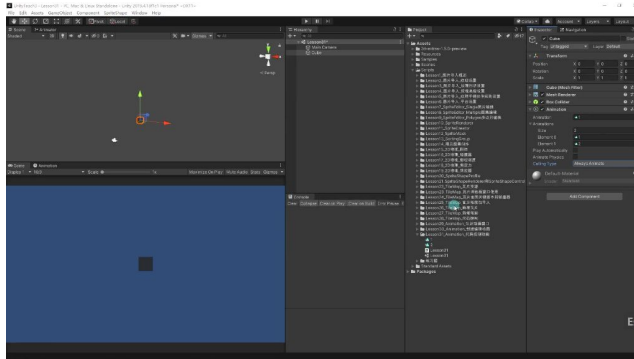


- 作用：决定是否在游戏开始时自动播放默认动画
- 使用场景：
 - 勾选时：运行即播放关联的默认动画
 - 取消勾选：需通过代码控制播放
- 注意：仅对已设置的默认动画有效

● animate Physics 07:10

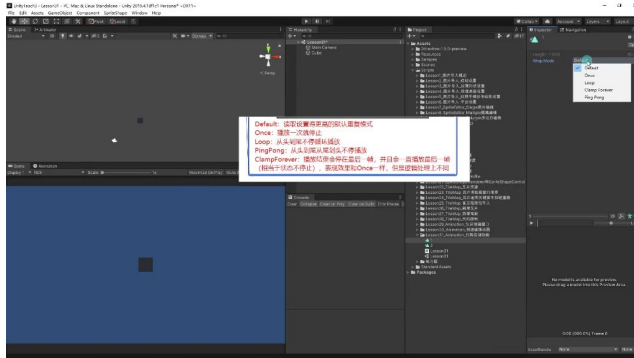
- 功能：控制动画是否与物理系统交互
- 实际应用：
 - 需要物理碰撞的动画可勾选
 - 多数开发场景中保持默认不勾选状态
- 特点：主要影响动画中的物理效果表现

● culling Type 07:28



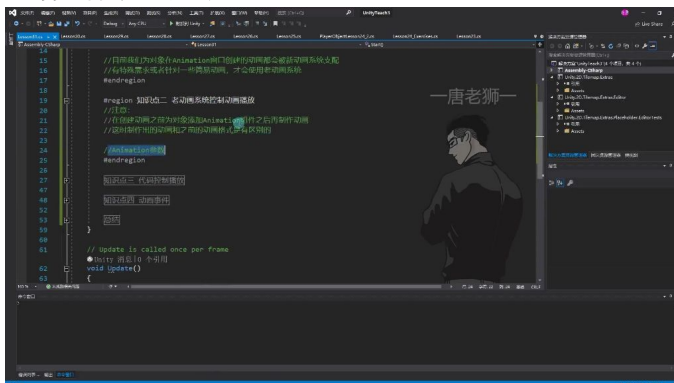
- 选项：
 - **Always Animate**：始终播放动画
 - **Based On Renderers**：基于渲染器状态决定
- 优化原理：
 - 当动画对象离开屏幕时可能停止播放
 - 使用默认动画姿势进行剔除判断
- 应用建议：根据性能需求选择合适的剔除方式

● wrap Mode 08:10

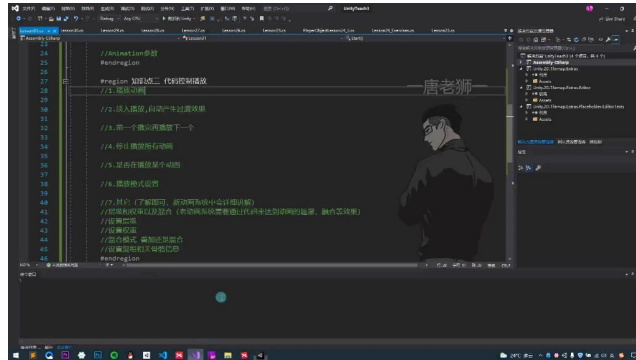


- 模式类型：
 - **Default**：读取默认重复模式（通常播放一次）
 - **Once**：播放一次停止（真正结束播放状态）
 - **Loop**：循环播放（如默认跑步动画）
 - **PingPong**：往返播放（类似乒乓球运动）
 - **ClampForever**：停在最后一帧（保持播放状态）
- 区别说明：
 - Once和ClampForever表现相似但逻辑状态不同
 - PingPong适合需要往复运动的动画效果
- 修改方法：在动画文件属性中设置Wrap Mode参数

4) 代码控制动画播放 09:55



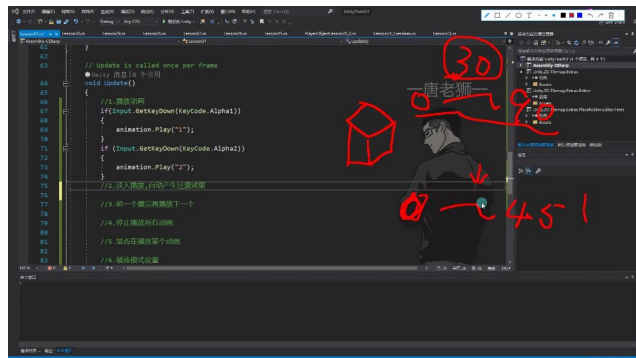
- 系统区别：Unity中有两套动画系统，新系统使用Animator组件控制动画，老系统使用Animation组件控制（主要用于Unity4之前版本或简易动画）
- 组件添加时机：必须在创建动画前为对象添加Animation组件，这样制作的动画格式与常规不同
- 播放动画 10:51



- 获取组件：通过GetComponent<Animation>()获取挂载的Animation组件
- 播放方法：animation.Play("动画名")可直接播放指定动画
- 示例代码：

```
1 if(Input.GetKeyDown(KeyCode.Alpha1)){
2     animation.Play("1"); //按1键播放动画1
3 }
```

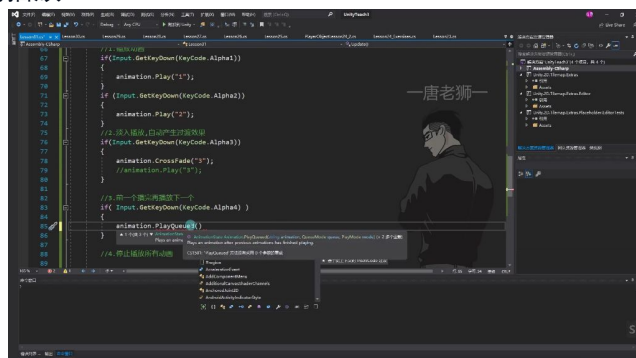
- 淡入播放 12:47



- 过渡效果：当新动画开始状态与当前状态不同时，自动计算中间值产生平滑过渡
- API对比：
 - CrossFade()会产生过渡效果（推荐）
 - 直接Play()会硬切到初始状态
- 典型场景：旋转30°时切换45°动画，CrossFade会先回正再旋转，而Play会直接重置

```
1 animation.CrossFade("3"); //按3键淡入播放动画3
```

- 队列播放 16:52

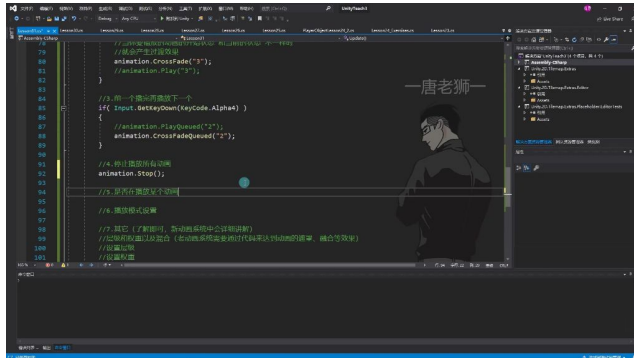


- 顺序播放：PlayQueued()让动画进入播放队列，前一个播完再播下一个

- 过渡版本: CrossFadeQueued()支持带过渡效果的队列播放
- 应用场景: 需要动画完整播放时使用, 避免中断当前动画

- 1 animation.PlayQueued("2"); //动画2进入队列
- 2 animation.CrossFadeQueued("2"); //带过渡的队列播放

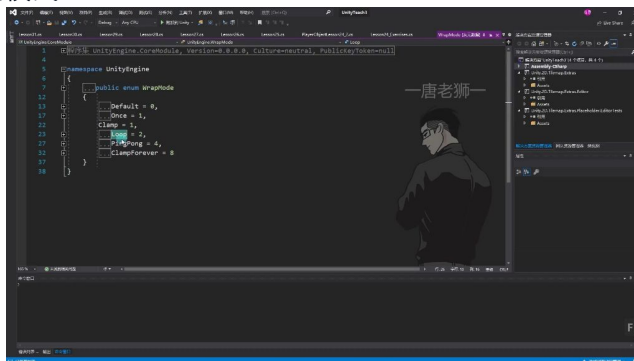
● 停止控制 20:53



- 全局停止: animation.Stop()停止所有正在播放的动画
- 状态判断: IsPlaying("动画名")检查特定动画是否在播放

- 1 animation.Stop(); //停止所有动画
- 2 if(animation.IsPlaying("1")){ /* 处理逻辑 */ }

● 播放模式 22:02

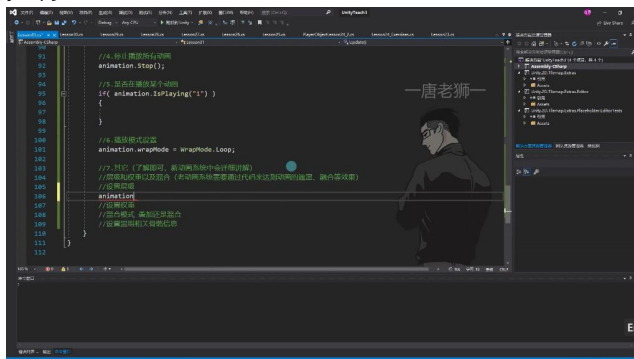


- 模式设置: 通过wrapMode属性控制播放行为
- 常用值:

- Loop: 循环播放
- Once: 播放一次
- PingPong: 往返播放

- 1 animation.wrapMode = WrapMode.Loop; //设置为循环模式

● 高级控制 22:38



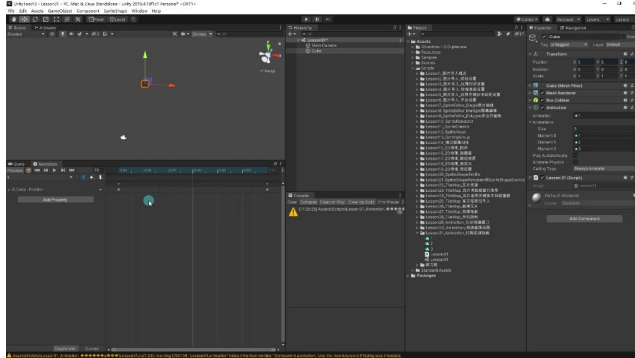
- 层级控制: animation["1"].layer设置动画层级
- 权重设置: animation["1"].weight (0-1范围)
- 混合模式:
 - Additive: 叠加模式

- Blend: 混合模式

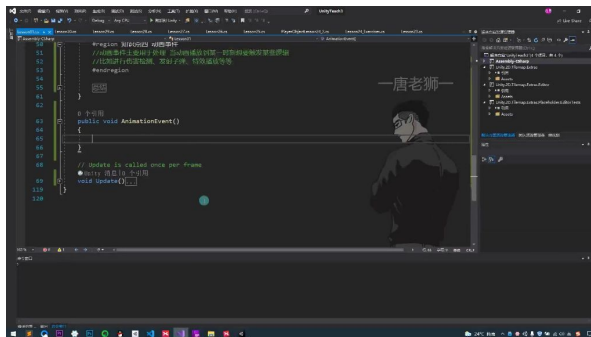
```
1 animation["1"].layer = 1;
2 animation["1"].weight = 1;
3 animation["1"].blendMode = AnimationBlendMode.Additive;
```

5) 动画事件 25:37

- 添加事件 26:06

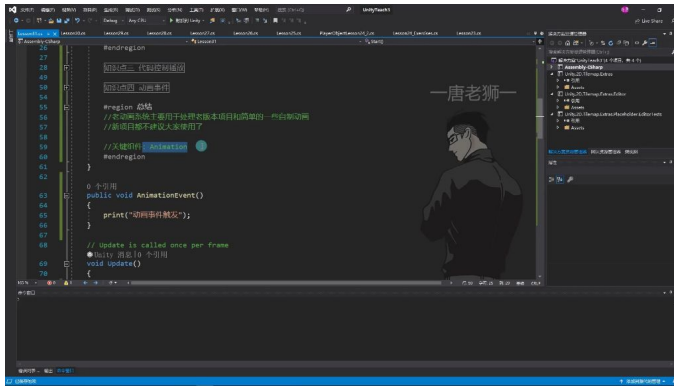


- 功能定义: 动画事件用于在动画播放到特定时间点时触发逻辑处理
- 应用场景:
 - 伤害检测
 - 发射子弹
 - 特效播放
 - 落地效果 (如灰尘特效)
 - 武器后坐力动画中的子弹发射时机
- 参数支持:
 - 支持int、float等基础类型
 - 支持GameObject等Unity对象类型
 - 可在Inspector窗口直接设置参数值

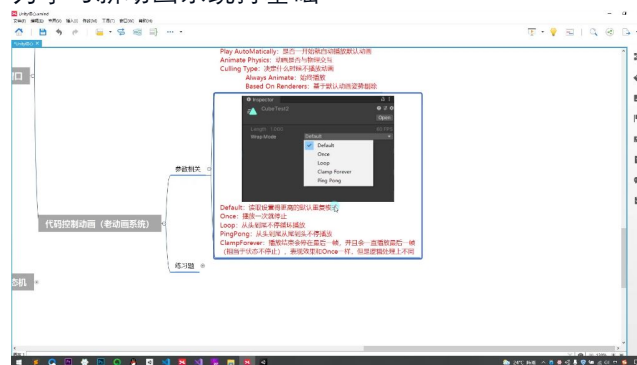


- 实现步骤:
 - 在脚本中创建public方法 (如public void AnimationEvent())
 - 在Animation窗口添加事件点 (蓝色标记点)
 - 选中事件点后在Inspector窗口绑定对应方法
 - 可拖动事件点调整触发时间
- 注意事项:
 - 方法必须为public才能被识别
 - 事件点可以添加多个, 对应不同时间点
 - 参数传递需在Inspector窗口设置具体值

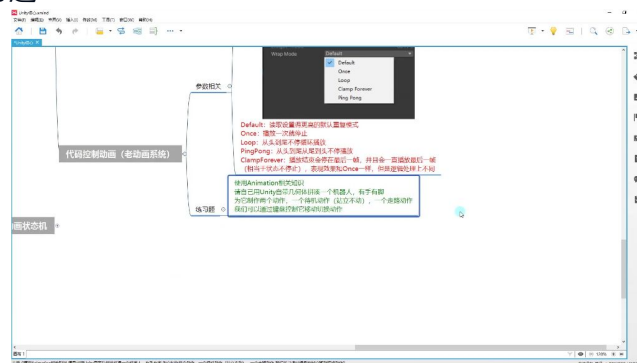
6) 老动画系统总结 31:18



- 核心组件：Animation组件
- 适用场景：
 - 处理老版本项目
 - 简单自制动画
- 新项目建议：不建议在新项目中使用
- 学习价值：
 - 理解动画系统基本原理
 - 能处理老项目或老资源
 - 为学习新动画系统打基础



- 播放模式：
 - Default：读取默认设置
 - Once：播放一次停止
 - Loop：循环播放
 - PingPong：往返播放
 - ClampForever：停在最后一帧但保持播放状态
- 练习题



- 练习要求：
 - 使用Unity基本几何体搭建机器人模型（需包含手脚）
 - 制作两个动画：
 - 待机动画（站立不动）

- 行走向动画
 - 通过键盘控制动作切换
- 实现要点：
 - 使用Animation组件控制动画
 - 掌握动画状态切换
 - 理解动画事件的应用

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
老动画系统	Unity中的旧版动画系统，使用Animation组件控制动画播放，适用于老版本项目或简单动画需求	与新动画系统（Animator组件）的区别：无状态机文件、动画格式不同	★★
代码控制动画播放	通过Animation.Play()播放动画、CrossFade()淡入过渡、PlayQueued()队列播放	Play() 与 CrossFade() 的区别：后者自动计算过渡效果，避免跳帧	★★★
动画事件	在动画时间轴添加事件，触发特定函数（如伤害检测、特效播放）	参数传递支持：int/float/GameObject等，需在Inspector中配置	★★★★
动画播放模式	设置动画循环（Loop）、单次（Once）、乒乓（PingPong）等模式	Default 模式与 Once 逻辑状态差异：后者彻底结束，前者停在末帧	★★
组件参数	Animation组件关键参数：默认动画、自动播放、物理交互、剔除规则	动画文件关联：需手动拖拽至Animations列表，新老系统不兼容	★★